ScribbleDB: Natural Language to Relational Database Synthesis

Aviroop Mitra
aviroopmitra@iisc.ac.in
Indian Institute of Science
Bengaluru, India

Anupam Sanghi
placeholder@iith.ac.in
Indian Institute of Technology Hyderabad
Sangareddy, India

ABSTRACT

[Placeholder ∼150 words. Motivate the greenfield database-generation problem: no real data, no formal spec, only a natural-language description. Introduce ScribbleDB as a four-stage agentic pipeline (fact extraction → schema generation → constraint modeling → code generation) whose output is a relational schema (DDL) plus an executable Python program that materializes the populated tables. Highlight the key new contributions versus prior workshop version: (i) deterministic inverse-function code generation for single-dependency DAG nodes, (ii) multivariate distribution support, (iii) ancestral sampling for logical constraints (FA = 1.0), (iv) context enrichment and shard-and-merge for scalability. Close with headline numbers: ∼6× table accuracy and ∼4× lower latency over SchemaAgent across three benchmark suites.]

CCS CONCEPTS

• **Information systems** → **Database design and models**; *Database utilities and tools.*

KEYWORDS

Database Generation; Schema Generation; Large Language Models; Agentic Pipelines; Ancestral Sampling; Constraint-based Synthesis

ACM Reference Format:
Aviroop Mitra and Anupam Sanghi. 2026. ScribbleDB: Natural Language to Relational Database Synthesis. In *Proceedings of ACM SIGMOD International Conference on Management of Data (SIGMOD '27).* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/XXXXXXX.XXXXXXX

1 INTRODUCTION

[Para 1 — Why synthetic databases matter: application development, testing, benchmarking; data-privacy and greenfield constraints prevent access to real data. Emphasise the greenfield setting: no existing deployment, no schema, no seed data.]

[Para 2 — State of the art and its limitations: ab-initio generators (MUDD, PDGF, Myriad) require formal declarative specs; regeneration-based tools (DBSynth, RSGen, Hydra, SAM) require production artefacts. Neither suits a non-technical domain expert with only an NL description.]

[Para 3 — LLMs as enabler and source of new challenges. Strong NL-to-SQL [18] and text-to-schema [15] capability. Three core challenges for full DB synthesis: ***(a) Expressiveness*** *(complex cross-table constraints),* ***(b) Hallucination*** *(error propagation across pipeline stages),* ***(c) Scalability*** *(context window degradation; materialization throughput).]*

[Para 4 — ScribbleDB overview: how it addresses each challenge. Emphasise that the output *is a DDL schema and an executable Python program, not tables directly—separating correctness verification from data materialization.]*

SIGMOD '27, 2027, TBD
2026. ACM ISBN XXX-X-XXXX-XXXX-X/27/00
https://doi.org/10.1145/XXXXXXX.XXXXXXX

Contributions.

- End-to-end NL-to-database agentic pipeline; outputs relational DDL schema + executable Python code for table materialization.
- *[New]* Deterministic code generation for single-dependency columns in the DAG via inverse functions on join/aggregate tables, reducing LLM calls and guaranteeing correctness-by-construction.
- *[New, partial]* Multivariate distribution support for cross-column correlations beyond univariate priors.
- Shard-and-merge schema generation with Gale–Shapley entity alignment.
- Ancestral sampling for logical constraint satisfaction; FA = 1.0 empirically.
- Comprehensive evaluation: ∼6× table accuracy, ∼4× lower latency vs. SchemaAgent [15] across three benchmark suites.

2 BACKGROUND AND RELATED WORK

2.1 Database Generation

[Para: Ab-initio generators (MUDD [12], PSDG [3], PDGF [9], Myriad [1]) operate from declarative specifications; powerful but require expert modeling effort. Regeneration-based approaches (DBSynth [8], RSGen [11], Hydra [10], SAM [17], DScaler [19], TouchStone [16]) exploit statistical metadata or workload traces from an existing deployment; inapplicable in greenfield settings. Neither class accepts NL input or handles joint distributional and logical constraints.]

2.2 LLM-based Schema and SQL Generation

[Para: NL-to-SQL benchmarks (Spider [18]) and the NL-to-SQL survey [6] demonstrate strong LLM capability on structured-output tasks. SchemaAgent / Text2Schema [15] is the closest prior work: it converts NL to a relational schema but stops short of data materialization, distributional constraints, and logical fidelity. ScribbleDB is the first system to cover the full synthesis pipeline.]

2.3 Agentic Systems for Structured Output

[Para: Multi-agent orchestration, self-repair loops, and tool-augmented LLMs have been applied to code generation, document understanding, and data engineering. ScribbleDB adopts these patterns but adds deterministic guard rails and a correctness-by-construction materialization scheme, distinguishing it from generation-only agentic approaches.]

3 PROBLEM FORMULATION

Definition 1 (Database Synthesis Problem). *[Formal definition box. Input: an NL description $\mathcal{D}$ of a target database domain plus a target row count n per fact table. Output: (i) a relational schema $\mathcal{S}$ (table definitions, primary keys, foreign keys, data types), and (ii)*

*[Figure 1 placeholder (full-width): Three panels. **Left**: NL description of the Credit Risk (Loan Origination) database. **Middle**: generated DDL schema excerpt (Applicants, Loans tables with PKs, FKs). **Right**: excerpt of rows from the materialized Loans table with the tiered interest-rate constraint annotated. Illustrates the end-to-end transformation.]*

Figure 1: ScribbleDB converts an NL description (left) into a relational DDL schema (middle) and an executable Python program whose output satisfies the stated constraints (right). Running example: Credit Risk (Loan Origination) database.

Table 1: Positioning ScribbleDB against representative prior systems. ✓ = full support, ○ = partial, × = not supported.

System	NL Input	Schema Gen.	Dist. Constr.	Logic Constr.	Materialize
MUDD / PDGF / Myriad	×	✓	✓	×	✓
DBSynth / RSGen / SAM	×	×	✓	×	✓
SchemaAgent	✓	✓	×	×	×
ScribbleDB	✓	✓	✓	✓	✓

a Python program $\mathcal{P}$ such that executing $\mathcal{P}$ produces a set of populated CSV/SQL tables satisfying $\mathcal{S}$ and all constraints implied by $\mathcal{D}$. Assumption: $\mathcal{D}$ is internally consistent (no contradiction detection).]

Correctness Criteria. *[Para: (a) Schema validity: PK uniqueness, FK referential integrity, data-type correctness. (b) Statistical fidelity: generated column distributions match those implied by $\mathcal{D}$ (measured by KS, MRE, NLL). (c) Logical fidelity: all decision-tree and aggregation constraints satisfied (measured by FA). (d) Completeness: recall of schema elements described in $\mathcal{D}$.]*

Running Example. *[Para: Introduce the* Credit Risk (Loan Origination) *database as the running example throughout the paper. Summarise the NL description: applicants with credit scores (300–850), annual income, DTI ratio; approved loans with an interest rate derived from a tiered lookup of credit score and loan amount (e.g., credit score > 750 & amount < \$50K ⇒ rate = 3.5%); overall interest rate follows a Gaussian distribution. This example is used in all stage-level figures and the case study in §9.]*

4 SYSTEM ARCHITECTURE

[Para 1: Four-stage pipeline overview: Fact Extraction → Schema Generation → Constraint Modeling → Code Generation. Each stage uses deterministic guard rails; failures trigger a bounded repair loop (max 5 retries per stage). The pipeline outputs DDL + a Python program, not tables directly, separating synthesis from materialization.]

[Para 2: Inter-stage interfaces. $\mathcal{F}$ = enriched fact set (JSON list of atomic statements with tags and external-kind annotations). $\mathcal{S}$ = relational schema (JSON: tables, columns, PKs, FKs, types). $\mathcal{C}$ = structured constraints (nested JSON: distributions, decision trees, aggregations). $\mathcal{G}$ = column-level dependency DAG (adjacency list). $\mathcal{P}$ = Python program (plain text).]

5 STAGE 1: FACT EXTRACTION

[Intro sentence: Stage 1 transforms the raw NL description $\mathcal{D}$ into a structured, enriched, and classified set of atomic facts $\mathcal{F}$ via a pipeline of LLM agents interleaved with deterministic validation and filtering steps.]

5.1 Atomic Fact Decomposition

[Para: A FactExtractor *LLM agent (tool-augmented: DuckDuckGo web-search) decomposes $\mathcal{D}$ into candidate* atomic facts*—minimal single-requirement statements (e.g., "An applicant has a credit score between 300 and 850"). Each fact carries: a unique identifier, its verbatim text, a list of* `referenced_fact_ids` *(other facts it depends on or elaborates), an* `is_external` *flag, and an origin span anchoring it to a substring of $\mathcal{D}$. The agent tracks previously errored fact IDs across repair iterations to avoid re-introducing known invalid facts.]*

5.2 Deterministic Fact Validation

[Para: After each extraction iteration the candidate set is subjected to a deterministic validator *that performs five correctness checks entirely without LLM involvement (Algorithm 1). Errors are typed (CRITICAL / MEDIUM / LOW) and fed back to the* FactExtractor *agent as a structured repair signal. Only CRITICAL errors trigger a re-extraction; MEDIUM and LOW errors are surfaced as warnings. Short origin spans (<10 characters) are downgraded to LOW rather than CRITICAL to avoid over-flagging terse but correct facts.]*

5.3 Integrity Verification

[Para: Once the fact set passes deterministic validation, a Verifier *LLM agent performs a semantic audit. It inspects the fact set for: (a)* missing information *(entities or constraints in $\mathcal{D}$ not yet captured as facts), (b)* introduced information *(facts that cannot be grounded in $\mathcal{D}$), (c)* changed constraints *(paraphrase drift from the original wording), and (d)* unresolved ambiguities*. The verifier emits a structured* `IntegrityReport`*; if non-empty, the* FactExtractor *agent is*

[Figure 2 placeholder (full-width): ScribbleDB architecture. Four stage boxes arranged left-to-right with labelled inter-stage arrows ($\mathcal{F}$, $\mathcal{S}$, $\mathcal{C}$+$\mathcal{G}$, $\mathcal{P}$). Inside each box: LLM agent icon(s) + deterministic validator block + repair-loop arrow. Final outputs annotated: DDL schema (SQL file) and Python materialization program.]

Figure 2: ScribbleDB architecture. Four stages connected by typed inter-stage interfaces. Each stage is guarded by a deterministic validator and a bounded repair loop. Final output: relational DDL schema and executable Python program.

Algorithm 1 Deterministic Fact Validator

Require: Candidate fact set F, source NL text $\mathcal{D}$
Ensure: List of typed error records E
1: *[Placeholder: normalize refs; check (1) invalid referenced IDs, (2) self-references, (3) DFS cycles on reference graph, (4) unverifiable origin via FactOriginMatcher (short spans → LOW severity), (5) external facts without any cited non-external fact; return typed errors CRITICAL / MEDIUM / LOW]*

re-invoked with the report appended to its context. This loop continues until the report is empty or the retry budget is exhausted.]

5.4 Fact Classification

[Para: After the verification loop, a Tagger *LLM agent performs a* one-shot *pass over the verified fact set. Each fact is tagged with one or more of four types:* Structural *(entity definitions, FK relationships),* Logical *(if–then rules, conditional constraints),* Statistical *(distributional parameters, value ranges), and* Metadata *(documentation, provenance notes). These tags are stored in the* `tags` *field of each* `AtomicFact` *and guide downstream agents: Schema Generation prioritises* Structural *and* Logical *facts; Code Generation consumes* Statistical *facts for distribution sampling.]*

5.5 Underspecification Detection and Context Enrichment

[Para: A deterministic underspecification detector *counts entities (<4), relationships, constraints, and total facts (<8) in the verified set and emits domain-specific DuckDuckGo search queries when the schema is likely under-described. These queries supplement those derived by the* ContextEnricher *agent.]*

[Para: The ContextEnricher *runs in two phases.* ***Phase 1 (pre-search)****: extract acronyms via regex* $[A-Z]\{2,5\}$*, combine with any suggestions from the* `IntegrityReport` *(up to 5 queries total), and execute DuckDuckGo searches via* `prefetch_and_format_searches`*; results are formatted as grounding context.* ***Phase 2 (LLM)****: the agent*

Algorithm 2 External Context Filter

Require: Accepted external facts F_{ext}, original non-external facts F_{orig}
Ensure: Classified, deduplicated external fact set F^*_{ext}
1: *[Placeholder: (1) dedup by normalised text; (2) discard self-referencing facts; (3) discard facts referencing IDs not in F_{orig}; (4) classify each fact into ExternalFactKind via keyword matching (TECHNICAL_DEFINITION | ARCHITECTURE_PATTERN | DOMAIN_PATTERN | DOMAIN_CONSTRAINT_HINT)]*

Algorithm 3 Stage 1: Fact Extraction & Enrichment

Require: NL description $\mathcal{D}$
Ensure: Enriched, tagged atomic fact set $\mathcal{F}$
1: *[Placeholder: FactExtractor → DetFactValidator loop (Alg. 1) → Verifier repair loop → Tagger (one-shot) → UnderspecDetector → ContextEnricher (web pre-search + LLM) → ContextAuditor loop → ExternalContextFilter (Alg. 2) → return $\mathcal{F} = F \cup F^*_{ext}$]*

receives the enriched context and the verified fact set and generates a list of external facts *(technical definitions, architecture patterns, domain constraints) to augment $\mathcal{F}$. A* ContextAuditor *LLM agent then reviews the proposed external facts, accepting or rejecting each with a reason; accepted facts are accumulated across iterations. Ablation shows* +10% *Table F1 from enrichment alone (§9).]*

5.6 External Context Filtering

[Para: The accepted external facts are passed through a deterministic external context filter *(Algorithm 2) before being merged into $\mathcal{F}$. The filter deduplicates, validates cross-references, and assigns a structured* `ExternalFactKind` *label to each accepted fact via keyword matching. The kind label informs how Schema Generation and Code Generation agents interpret and weight the fact.]*

[Figure 3 placeholder: Credit Risk NL paragraph (top); 8–10 atomic facts extracted (middle), each annotated with its type tag (STRUCTURAL / LOGICAL / STATISTICAL / METADATA); one external fact with DTI definition inlined (bottom, highlighted) showing its ExternalFactKind = DOMAIN_CONSTRAINT_HINT.]

Figure 3: Fact extraction for the Credit Risk running example. Atomic facts are tagged by type; external facts carry a kind label.

6 STAGE 2: SCHEMA GENERATION

[Intro sentence: Stage 2 maps the enriched fact set $\mathcal{F}$ to a globally consistent relational schema $\mathcal{S}$ via a shard-and-merge strategy augmented with domain intelligence, deterministic validation, and a multi-agent review loop.]

6.1 Domain Intelligence Extraction

[Para: Before fact clustering, a DomainIntelligenceExtractor *one-shot LLM agent is invoked with the domain name and the analytical goal implicit in $\mathcal{F}$. It produces a* `DomainIntelligenceReport`*—a structured summary of typical entity types, common relationship patterns, and domain-specific design conventions for the target domain (e.g., normalisation conventions in healthcare, regulatory FK structures in finance). This report is passed as additional context to the* SchemaArchitect *agents in each shard and to the* DomainAuditor *loop (§6.5), anchoring the schema in domain expectations rather than solely in the facts.]*

6.2 Fact Clustering and Parallel Shard Generation

[Para: A Chunker *LLM agent partitions $\mathcal{F}$ into a* `ChunkedPlan`*: a list of* chunks *(each a labeled cluster of fact IDs covering one cohesive sub-domain) plus a* `core_modeling_facts` *set of globally applicable facts (e.g., cross-table FK conventions, domain-wide cardinality rules) that are broadcast to every shard. The Chunker participates in a deterministic repair loop: structural errors in the proposed plan (e.g., orphaned fact IDs, empty chunks) are fed back as typed error signals.]*

[Para: Each chunk is processed in parallel by a SchemaArchitect *LLM agent that receives the chunk's facts, the* `core_modeling_facts`*, and the* `DomainIntelligenceReport`*. It produces a candidate schema shard: table definitions, columns, PKs, and intra-shard FKs. A deterministic repair loop (Algorithm 4) guards each shard; the agent maintains a* `fix_history` *to avoid re-introducing previously repaired errors.]*

6.3 Deterministic Shard Validation

[Para: Each shard undergoes a pure-algorithmic check before merging: PK uniqueness within the shard, FK referential completeness (all FK targets exist in the shard or are flagged as cross-shard pending), and schema-graph connectivity (no isolated table). Failures emit a typed repair signal to the schema-architect agent; no LLM call within the validator itself.]

Algorithm 4 Deterministic Shard Validation

Require: Schema shard S_i
Ensure: Validated S_i or typed repair signal
1: *[Placeholder: (1) PK uniqueness per table (PK_CLASH); (2) FK target existence or cross-shard pending flag (FK_DANGLE); (3) BFS connectivity—no isolated table (ISOLATED_TABLE); emit typed repair signal to SchemaArchitect on failure]*

Algorithm 5 Deterministic Schema Merge via Stable Matching

Require: Validated shards $\{S_1, \ldots, S_k\}$, threshold θ
Ensure: Merged global schema $\mathcal{S}$, `MergeDecisionLog`
1: *[Placeholder: Pass 1—pairwise $\sigma = \alpha\,\sigma_{name} + (1-\alpha)\,\sigma_{attr}$ scoring (embedding cosine + lexical fallback; column-pair GS pass) and Gale–Shapley stable matching; Pass 2—FK consistency prevalidation per matched pair; Pass 3—column union + relationship remapping; post-merge repairs: junction consolidation, relationship name repair, FK type alignment, orphan FK wiring, connectivity check]*

6.4 Shard Merging via Stable Matching

[Para: Independently generated shards may contain overlapping representations of the same entity. The SchemaMerger *is a fully deterministic three-pass procedure (Algorithm 5) that identifies and resolves these overlaps.]*

[Para: ***Similarity scoring.*** *For each candidate table pair (T_a, T_b) across shards, a composite score $\sigma = \alpha \cdot \sigma_{name} + (1-\alpha) \cdot \sigma_{attr}$ is computed. σ_{name} is the cosine similarity of sentence-transformer embeddings (all-MiniLM-L6-v2) with a lexical fallback: $\sigma_{lex} = 0.3 \cdot J + 0.7 \cdot O$ (Jaccard J, context-overlap O); the final name score is $\max(\sigma_{lex}, \sigma_{emb})$, using the strongest available signal. σ_{attr} is a second Gale–Shapley [2] pass over column pairs. Shard pairs with $\sigma > \theta$ are candidate overlaps.]*

6.5 Post-Merge Review and Domain Audit

[Para: After merging, a MergeReviewer *LLM agent reviews the merged schema $\mathcal{S}$, the* `MergeDecisionLog`*, and the original per-shard facts. It identifies merge decisions that appear semantically inconsistent (e.g., two tables with different semantics merged due to a high name-similarity score) and proposes targeted patches.]*

[Para: A DomainAuditor *LLM loop agent then performs a deeper structural audit using the* `DomainIntelligenceReport`*, the full fact clusters, and any structural errors emitted by a deterministic patch validator. It produces a* `CritiqueReport` *with typed patches (add table, rename column, add FK, etc.); each patch is validated deterministically before being applied. The loop continues until the* `CritiqueReport` *is empty or the retry budget is exhausted.]*

[Para: Finally, a ComplianceCertifier *LLM agent performs a one-shot analytical-utility check—verifying that $\mathcal{S}$ can express the*

[Figure 4 placeholder: Two schema shards (left) each containing a variant of the Applicant/Customer entity. Centre: Gale–Shapley matching arrow annotated with σ_{name} and σ_{attr} scores and the FK consistency verdict. Right: merged global schema with unified Applicant table, resolved cross-shard FK, and the four post-merge repair steps annotated.]

Figure 4: Shard-and-merge for Credit Risk. Overlapping Applicant entities are detected by stable matching and unified via the three-pass merge procedure.

Algorithm 6 Stage 2: Schema Generation

Require: Fact set $\mathcal{F}$
Ensure: Global schema $\mathcal{S}$
1: *[Placeholder: DomainIntelligenceExtractor (one-shot LLM) → Chunker + det. loop → parallel SchemaArchitect + ShardValidation (Alg. 4) → SchemaMerge (Alg. 5) → MergeReviewer (LLM) → DomainAuditor loop + det. patch validation → ComplianceCertifier (one-shot LLM) → return $\mathcal{S}$]*

analytical queries implied by $\mathcal{D}$ (e.g., that columns needed to compute portfolio-level aggregates are present). The certifier's verdict is informational; it does not block pipeline execution but is logged for user inspection. This produces the final validated global schema $\mathcal{S}$.]

7 STAGE 3: CONSTRAINT MODELING

7.1 Structured Constraint Representation

[Para: All constraints are stored in a nested structured format (JSON): (a) univariate distributions (type, parameters, target column); (b) decision-tree logical constraints (if–then rules over column values, including cross-table aggregations such as SUM, MAX); (c) multivariate distribution groups (§7.3). This representation enables downstream deterministic validation and code generation.]

7.2 Constraint Extraction and Feasibility Verification

[Para: The global schema graph is sharded into connected components. A constraint-modeling agent extracts structured constraints per component in parallel. A mathematics agent then verifies distribution feasibility (e.g., checking that parameter values are in the valid domain for the specified distribution family). Decision-tree feasibility is checked deterministically (Algorithm 7).]

7.3 Multivariate Distribution Support

[Para — PLACEHOLDER (not fully fleshed out): Extension beyond univariate constraints to capture cross-column correlations. Planned approach: [TBD — likely copula-based joint sampling (Gaussian or vine copulas) or a conditional chain representation where later columns in the DAG topological order are sampled conditionally on earlier ones]. Describe: (i) the structured representation for multivariate groups in C; (ii) how multivariate group nodes are inserted into the dependency graph $\mathcal{G}$; (iii) how the code generator handles a multivariate group node. This section will expand substantially once the approach is finalised.]

Algorithm 7 Deterministic Decision-Tree Feasibility Check

Require: Set of decision-tree constraints $T \subseteq C$
Ensure: Feasibility verdict per constraint; infeasibility report if any
1: *[Placeholder: per constraint—enumerate leaf conditions, check contradictory conjunctions (e.g., $x > 750$ AND $x < 300$), verify outcome type compatibility, check referenced columns exist in $\mathcal{S}$; flag infeasible constraints for LLM repair]*

Algorithm 8 Deterministic Dependency DAG Construction

Require: Schema $\mathcal{S}$, structured constraints C
Ensure: Column-level DAG $\mathcal{G}$
1: *[Placeholder: one node per column; FK edges reverse-causal; constraint edges outcome ← driver; multivariate group nodes (TBD); topological sort; cycle → patch agent updates C, rebuild]*

[Figure 5 placeholder: Column-level DAG for Credit Risk. Solid nodes: `credit_score` (leaf), `loan_amount` (leaf), `interest_rate` (root). Solid directed edges: `credit_score` → `interest_rate`, `loan_amount` → `interest_rate`. FK edges shown as dashed. Dashed composite node group for a multivariate dependency (placeholder). Node colouring: deterministic-path nodes (blue), LLM-path nodes (orange).]

Figure 5: Dependency DAG for the Credit Risk running example. `interest_rate` is the reverse-causal root; its drivers are leaves. Node colour indicates the code-generation path taken in Stage 4.

7.4 Dependency Graph Construction

[Para: From FK links and structured constraints, a column-level DAG $\mathcal{G}$ is built in reverse-causal *order: constrained outcome columns (e.g.,* `interest_rate`*) are positioned as roots; their independent drivers (e.g.,* `credit_score`, `loan_amount`*) are leaves. Multivariate group nodes are inserted as composite nodes (TBD). Cycle detection is performed via topological sort; if a cycle is detected, a patch agent updates C to break the cycle and the graph is rebuilt.]*

Algorithm 9 Stage 3: Constraint Modeling

Require: Schema $\mathcal{S}$, fact set $\mathcal{F}$
Ensure: Structured constraints C, dependency DAG $\mathcal{G}$
1: *[Placeholder: shard $\mathcal{S}$ into connected components → parallel constraint-modeling agents → math agent distribution feasibility → DTFeasibilityCheck (Alg. 7) → BuildDAG (Alg. 8) → return $C, \mathcal{G}$]*

Algorithm 10 Deterministic Inverse-Function Code Synthesis

Require: Single-dependency column c, aggregation type τ, constraint ϕ
Ensure: Python code snippet p_c
1: *[Placeholder: synthesise intermediate aggregate/join table; dispatch on τ: SUM → uniform partition, MAX → argmax selection, AVG → constrained sampling, JOIN → FK-key draw; unsupported τ falls through to LLM path (§8.2)]*

8 STAGE 4: CODE GENERATION

8.1 Deterministic Inverse-Function Code for Single-Dependency Columns

[Para: Key new contribution. For each column node $c \in \mathcal{G}$ with exactly one *incoming dependency edge, Python code is synthesised* deterministically*—no LLM call is made. The strategy is: (i) synthesise an intermediate join or aggregate table satisfying the aggregate constraint (the "forward" computation); (ii) decompose back to base column values using an inverse function whose form is fully determined by the aggregation type. This guarantees correctness-by-construction for the covered columns and eliminates LLM API cost for these nodes. Covered aggregation types and their inverses: SUM → uniform partition, MAX → argmax selection, AVG → constrained sampling around mean, JOIN (FK assignment) → deterministic FK-key draw.]*

8.2 LLM Ancestral Sampling for Intertwined Constraints

[Para: When a column node has multiple incoming dependency edges—i.e., its value is governed by constraints involving several other columns simultaneously—no clean closed-form inverse exists. In this case, a code-generator LLM agent is invoked. The agent receives the local sub-DAG centred on c, the relevant constraints from C, and the pre-determined generation order from $\mathcal{G}$. It synthesises an inverse function *generatively: given already-sampled values for the driver columns, it emits Python code that samples c such that the constraints are satisfied. By following the reverse-causal order of $\mathcal{G}$, this scheme guarantees constraints are satisfied* by construction*, with no post-hoc repair.]*

8.3 Code Merge and Smoke Testing

[Para: Shard-level Python snippets are merged into a single program using a fixed code template with five sections: imports, seed definitions, helper functions, intermediate-table synthesis, base-table materialisation. When two snippets contain conflicting initialisations of the same column (e.g., one uses a distribution spec, another uses random initialisation), a deterministic tie-breaking rule resolves the conflict

Algorithm 11 LLM Ancestral Sampling for Intertwined Constraints

Require: Intertwined column c, its local sub-DAG, constraints $C_c \subseteq C$
Ensure: Python code snippet p_c
1: *[Placeholder: build prompt (sub-DAG + C_c + generation order + template) → LLM synthesises inverse function → deterministic sanity check (type compatibility, no undefined variables) → repair loop max 5 retries]*

[Figure 6 placeholder: Decision flowchart. Input: column node c from $\mathcal{G}$. Diamond: "single incoming dependency?" Yes → Alg. 10 (deterministic inverse, labelled with aggregation type). No → Alg. 11 (LLM ancestral sampling). Illustrated on Credit Risk DAG: `interest_rate` *routed to LLM (two drivers);* `loan_amount` *FK reference routed to deterministic JOIN inverse.]*

Figure 6: Code generation routing per column node in $\mathcal{G}$. Single-dependency columns take the deterministic inverse path (no LLM call); intertwined columns are handled by the LLM ancestral-sampling agent.

Algorithm 12 Deterministic Code Merge

Require: Shard-level code snippets $\{p_1, \ldots, p_k\}$, code template $\mathcal{T}$
Ensure: Consolidated Python program $\mathcal{P}$
1: *[Placeholder: align snippets to template sections (imports / seeds / helpers / intermediate / base); resolve column initialisation conflicts via tie-breaking rules; smoke test at 10% scale; repair loop max 5 retries on failure]*

Algorithm 13 Stage 4: Code Generation

Require: Schema $\mathcal{S}$, constraints C, DAG $\mathcal{G}$
Ensure: Python program $\mathcal{P}$
1: *[Placeholder: traverse $\mathcal{G}$ in reverse-topological order; single in-edge → DetInverseCode (Alg. 10), multiple in-edges → LLMAncestralSampling (Alg. 11); CodeMerge (Alg. 12) → return $\mathcal{P}$]*

(distribution-spec ≻ random; structured ≻ unstructured). The merged program is smoke-tested by running it at 10% of the target row count. Any runtime failure triggers an agent repair loop before full-scale materialisation. The final output is $\mathcal{P}$.]

9 EXPERIMENTAL EVALUATION

9.1 Experimental Setup

Datasets. *[Placeholder: (a) RSchema [15]: 381 NL-to-schema test cases (schema only, no data). (b) Handcrafted [7]: 135 test cases (NL,*

Table 2: Schema quality comparison. Best result per metric/-dataset in bold.

Dataset	System	TF1	TAcc	AF1	AAcc	PKAcc	DTAcc
RSchema	SchemaAgent				*[values]*		
	ScribbleDB				*[values]*		
Handcrafted	SchemaAgent				*[values]*		
	ScribbleDB				*[values]*		
Benchmark	SchemaAgent				*[values]*		
	ScribbleDB				*[values]*		

Table 3: Data modeling quality. Metrics penalised by schema recall fraction. *[TBD: multivariate column to be added once §7.3 is finalised.]*

Dataset	MRE↓	NLL↑	KS↓	FA↑
Handcrafted		*[values]*		
TPC-H		*[values]*		
TPC-DS		*[values]*		
IMDB		*[values]*		
MIMIC-IV		*[values]*		

schema, distributional and logical constraints) generated via Claude 4.5 Sonnet, spanning Credit Risk, Quantitative Finance, Clinical Trials; 3–50 tables; 10K–1M rows per fact table. 2:3:5 ratio across small (<10), medium (11–25), large (26+) schemas. (c) Benchmarks: IMDB [4], TPC-H [13], TPC-DS [14], MIMIC-IV [5] official documentation converted to NL descriptions that deliberately omit benchmark names, specific schema details, and explicit parameters.]

LLM & Baseline. *[Placeholder: GPT-4o for all LLM components of ScribbleDB. Baseline for schema quality: SchemaAgent [15]. No prior baseline for data modeling quality (ScribbleDB is first to target full synthesis).]*

Metrics. *[Placeholder: Schema: Table/Attr/PK/DT F1 and Accuracy. Statistical fidelity: MRE↓, NLL↑, KS↓. Logical fidelity: FA↑. Efficiency: schema gen tokens/time, per-row gen time, avg. retry loops. New metric:* Deterministic Path Ratio *(§9.7) — fraction of DAG nodes routed to deterministic vs. LLM code generation.]*

9.2 Schema Quality

[Placeholder: ScribbleDB achieves near-perfect Table F1 (95–100) across all three dataset groups. SchemaAgent degrades sharply on complex schemas (Table F1 drops to 48–52 on Handcrafted/Benchmark). Key insight: sharding enables parallel agents to focus on smaller schema portions; combined with context enrichment, it recovers schema elements that SchemaAgent misses when the NL is ambiguous or incomplete.]

9.3 Data Modeling Quality

[Placeholder: Statistical realism (KS < 0.3, MRE < 0.2, NLL > 0.7) and logical fidelity (FA = 1.0) across Handcrafted and Benchmark datasets. Metrics are schema-recall-penalised to account for structural mismatches. Note: multivariate evaluation metrics will be added in this table once §7.3 is finalised—a new column (e.g., copula KS or rank correlation error) is reserved.]

[Figure 7 placeholder: Three panels (one column each). Each: generated histogram (blue) overlaid with ground-truth density curve (orange). Panel 1: normally distributed column (e.g., income). Panel 2: Zipfian column (e.g., transaction count). Panel 3: bounded column (e.g., credit score 300–850). All from Handcrafted dataset.]

Figure 7: Distribution fidelity: generated vs. ground-truth density for three representative columns in the Handcrafted dataset.

Table 4: Ablation on Handcrafted dataset. Best result per column in bold.

Configuration	TF1	TAcc	AF1	MRE	NLL	KS	FA
Full ScribbleDB				*[values]*			
w/o Enrichment				*[values]*			
w/o Sharding				*[values]*			
w/o Anc. Sampling				*[values]*			
w/o Det. CodeGen				*[values — new row]*			

Table 5: Cost and efficiency (avg. per test case).

Dataset	System	Tokens	Sch. Time	Row Time	Retries
RSchema	SchemaAgent		*[values]*		
	ScribbleDB		*[values]*		
Handcrafted	SchemaAgent		*[values]*		
	ScribbleDB		*[values]*		
Benchmark	SchemaAgent		*[values]*		
	ScribbleDB		*[values]*		

9.4 Scalability Analysis

[Placeholder: Schema-generation latency scales sub-linearly with table count for ScribbleDB (parallel sharding) vs. linearly for SchemaAgent (sequential). 3–4× speedup on complex schemas. Row-generation time scales linearly with target row count at 10–30 ms/row, independent of schema complexity (deterministic and LLM code paths both amortise to per-row costs).]

9.5 Ablation Studies

[Placeholder: Quantify contribution of each architectural component on the Handcrafted dataset. New ablation row: "w/o Det. CodeGen" replaces all deterministic code-generation paths with LLM calls—expected to show higher token cost, higher latency, and potentially lower FA if the LLM occasionally fails to satisfy constraints exactly.]

*[Figure 8 placeholder (full-width, two panels). **Left**: schema generation latency (s) vs. number of tables. Two lines: ScribbleDB (with 1σ shading over test cases) and SchemaAgent. **Right**: per-row generation time (ms) vs. target row count n for Handcrafted dataset; separate lines for deterministic-only nodes and LLM nodes.]*

Figure 8: Scalability. Left: ScribbleDB's parallel sharding yields 3–4× lower latency than SchemaAgent on large schemas. Right: per-row generation time is constant across row counts.

[Figure 9 placeholder: Stacked bar chart. X-axis: schema-size bins (small, medium, large). Y-axis: fraction of DAG column nodes. Two stacks: deterministic-path nodes (blue) and LLM-path nodes (orange). Show that larger schemas have higher DPR (more single-dependency FK columns).]

Figure 9: Deterministic Path Ratio by schema size. The fraction of columns handled deterministically grows with schema size, amplifying the LLM call reduction on large schemas.

9.6 Cost and Efficiency

9.7 Deterministic Code Generation Analysis

[Placeholder: Characterise the new deterministic code-generation contribution. Metrics to report: (a) Deterministic Path Ratio (DPR)*: fraction of DAG column nodes routed to Alg. 10 vs. LLM (Alg. 11). Expect DPR ≈ 40–60% on Handcrafted dataset (hypothesis: many FK and simple aggregate columns are single-dependency). (b) LLM call reduction: absolute and relative number of LLM calls saved vs. an all-LLM baseline. (c) Correctness delta: compare FA for deterministic-path columns vs. LLM-path columns— deterministic path expected to achieve FA = 1.0 by construction.]*

9.8 Case Study: Credit Risk Database

[Placeholder: End-to-end walkthrough. NL description (§5) → 9 atomic facts tagged STRUCTURAL/LOGICAL/STATISTICAL (Stage 1) → DDL schema: Applicants (ApplicantID PK, CreditScore, Income, DTI), Loans (LoanID PK, ApplicantID FK, LoanAmount, InterestRate) (Stage 2) → dependency DAG: `interest_rate` ← `credit_score`, `loan_amount` *(Stage 3) → generated Python code:* `interest_rate` *routed to LLM ancestral-sampling (two drivers);* `loan_amount` *FK column routed to deterministic JOIN inverse (Stage 4) → sample of materialized rows verifying tiered-lookup constraint.]*

Table 6: Sample generated rows: Credit Risk, Loans table. `IntRate` satisfies the tiered-lookup constraint by construction.

LoanID	AppID	CreditScore	LoanAmt	IntRate
[placeholder rows with IntRate satisfying tiered-lookup]				

Listing 1: Python code excerpt generated by ScribbleDB for the Credit Risk database. The `sample_interest_rate` function implements the LLM-synthesised inverse for the tiered-lookup constraint.

```
# [Placeholder: Python code excerpt]
# Expected content:
#   def sample_interest_rate(credit_score,
    loan_amount):
#       """Inverse function synthesised by LLM
    agent (Alg. 11).
#       Implements tiered lookup: credit_score >
     750 and amount < 50000 -> rate = 0.035
#       ... additional tiers ...
#       """
#       ...
#
#   def sample_loan_amount(applicant_ids):
#       """Deterministic FK-assignment inverse (
    Alg. 10, JOIN case)."""
#       return np.random.choice(applicant_ids,
    size=N_LOANS, replace=True)
```

10 DISCUSSION

10.1 Limitations

[Placeholder: (1) Assumes NL input is internally consistent; no contradiction detection or resolution. (2) Multivariate distribution support is partial—see §7.3; copula or conditional-chain design not yet finalised. (3) Deterministic inverse synthesis covers only single-dependency columns; many-to-many joins and compound multi-column aggregations fall back to LLM. (4) Context enrichment relies on web retrieval quality; domain jargon absent from public knowledge bases may still be under-specified.]

10.2 Failure Modes

[Placeholder: (a) Overly ambiguous NL where enrichment cannot recover missing distribution parameters (e.g., no public definition of a proprietary scoring model). (b) DAG cycles introduced by contradictory logical constraints that the patch agent cannot resolve within the retry budget. (c) Distribution parameter hallucination for rare distribution families (e.g., generalised extreme value distributions). (d) Smoke-test failures that cannot be repaired by the code-generation agent within 5 retries—currently causes a pipeline abort.]

10.3 Future Directions

[Placeholder: (1) Multivariate distributions: copula-based joint sampling to model cross-column correlations beyond univariate priors; Gaussian copulas as a first step. (2) Contradiction detection and resolution in NL input before fact extraction. (3) Extending deterministic code generation to multi-dependency chains (iterative inverse composition for chained aggregations). (4) Schema evolution: incrementally updating the generated schema and program as the NL description changes, without full pipeline re-execution. (5) Support for non-relational output (document stores, graph databases).]

11 CONCLUSION

[Placeholder ∼100 words. Summarise ScribbleDB: a four-stage agentic pipeline converting an NL description into a validated relational DDL schema and an executable Python program for large-scale table materialization. Restate the key new contributions over the prior workshop version: deterministic inverse-function code generation for single-dependency DAG nodes and multivariate distribution support (in progress). Restate headline experimental results. Close with one forward-looking sentence on copula-based multivariate modelling and contradiction handling as the next milestones.]

ACKNOWLEDGMENTS

[Placeholder: Thank Jayant Haritsa (valuable feedback and guidance), Carsten Binnig (discussions that inspired the problem statement), and Nitthesh D and Udit Senapaty (implementation support and testing).]

REFERENCES

[1] Alexander Alexandrov, Kostas Tzoumas, and Volker Markl. 2012. Myriad: Scalable and Expressive Data Generation. *Proc. VLDB Endow.* 5, 12 (2012), 1890–1893. https://doi.org/10.14778/2367502.2367530

[2] D. Gale and L. S. Shapley. 1962. College Admissions and the Stability of Marriage. *The American Mathematical Monthly* 69, 1 (1962), 9–15. https://doi.org/10.1080/00029890.1962.11989827

[3] Joseph E. Hoag and Craig W. Thompson. 2007. A Parallel General-Purpose Synthetic Data Generator. *SIGMOD Rec.* 36, 1, 19–24. https://doi.org/10.1145/1276301.1276305

[4] IMDb. 2024. IMDb Non-Commercial Datasets. https://datasets.imdbws.com Accessed: 2026-05-08.

[5] Alistair Johnson, Lucas Bulgarelli, Tom Pollard, et al. 2024. MIMIC-IV. https://doi.org/10.13026/kpb9-mt58 Version 3.1.

[6] Yuyu Luo, Guoliang Li, Ju Fan, Chengliang Chai, and Nan Tang. 2025. Natural Language to SQL: State of the Art and Open Problems. *Proc. VLDB Endow.* 18, 12 (2025), 5466–5471. https://doi.org/10.14778/3750601.3750696

[7] Aviroop Mitra. 2026. Autonomous-Data-Architect. https://github.com/Aviroop07/Autonomous-Data-Architect GitHub repository.

[8] Tilmann Rabl, Manuel Danisch, Michael Frank, Sebastian Schindler, and Hans-Arno Jacobsen. 2015. Just Can't Get Enough: Synthesizing Big Data. In *Proc. ACM SIGMOD*. 1457–1462. https://doi.org/10.1145/2723372.2735378

[9] Tilmann Rabl, Michael Frank, Hatem Mousselly Sergieh, and Harald Kosch. 2010. A Data Generator for Cloud-Scale Benchmarking. In *Proc. TPCTC*. 41–56.

[10] Anupam Sanghi, Raghav Sood, Jayant R. Haritsa, and Srikanta Tirthapura. 2018. Scalable and Dynamic Regeneration of Big Data Volumes. In *Proc. EDBT*. 301–312. https://doi.org/10.5441/002/EDBT.2018.27

[11] Entong Shen and Lyublena Antova. 2013. Reversing Statistics for Scalable Test Databases Generation. In *Proc. DBTest*. https://doi.org/10.1145/2479440.2479445

[12] John M. Stephens and Meikel Poess. 2004. MUDD: A Multi-Dimensional Data Generator. In *Proc. WOSP*. 104–109. https://doi.org/10.1145/974044.974060

[13] Transaction Processing Performance Council. 2022. TPC Benchmark™ H Standard Specification. https://www.tpc.org/tpch/ Version 3.0.1.

[14] Transaction Processing Performance Council. 2024. TPC Benchmark™ DS Standard Specification. https://www.tpc.org/tpcds/ Version 4.0.0.

[15] Qin Wang, Youhuan Li, Yansong Feng, et al. 2025. Text2Schema: Filling the Gap in Designing Database Table Structures based on Natural Language. arXiv:2503.23886

[16] Qingshuai Wang, Yuming Li, Rong Zhang, et al. 2023. A Scalable Query-Aware Enormous Database Generator for Database Evaluation. *IEEE Trans. Knowl. Data Eng.* 35, 5 (2023), 4395–4410. https://doi.org/10.1109/TKDE.2022.3153651

[17] Jingyi Yang, Peizhi Wu, Gao Cong, Tieying Zhang, and Xiao He. 2022. SAM: Database Generation from Query Workloads with Supervised Autoregressive Models. In *Proc. ACM SIGMOD*. 1542–1555. https://doi.org/10.1145/3514221.3526168

[18] Tao Yu, Rui Zhang, Kai Yang, et al. 2019. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. arXiv:1809.08887

[19] J. W. Zhang and Y. C. Tay. 2016. DScaler: Synthetically Scaling a Given Relational Database. *Proc. VLDB Endow.* 9, 14 (2016), 1671–1682. https://doi.org/10.14778/3007328.3007333